Pricing European and American Vanilla Options Using the Binomial Model

Hang Yu (hangy6@illinois.edu)

Summer 2024

Team Members

- Hang Yu: Project planning, code implementation, data analysis, report writing.

1 Introduction

Options are financial derivatives that provide the right, but not the obligation, to buy or sell an underlying asset at a specified strike price before or at the expiration date. This project focuses on pricing European and American vanilla options using the binomial model and implementing the binomial de-Americanization method.

1.1 Objective

The objective of this project is to implement the Cox-Ross-Rubinstein (CRR) binomial model to price European and American puts and calls on a stock paying continuous dividend yield. Additionally, the project investigates the convergence of the binomial model to the Black-Scholes model for European options and explores the early exercise boundary for American options.

1.2 Overview

- Section 1: Introduction
- Section 2: Implementation and Results
- Section 3: Discussion and Conclusion
- Appendix: Code Implementation

2 Implementation and Results

This section describes the implementation details of the CRR binomial model and the experiment results.

2.1 Implementation of the CRR Binomial Model

The binomial model is implemented to price European and American options on a stock paying continuous dividend yield. The key parameters are the strike price (K), time to maturity (T), initial stock price (S0), volatility (σ), risk-free interest rate (r), dividend yield (q), number of time steps (N), and option type (Exercise).

2.1.1 Model Parameters

$$\delta = \frac{T}{N}$$
$$u = e^{\sigma\sqrt{\delta}}$$
$$d = \frac{1}{u} = e^{-\sigma\sqrt{\delta}}$$
$$p = \frac{e^{(r-q)\delta} - d}{u - d}$$

p is the risk neutral probability.

2.1.2 Stock Price Tree

The stock price tree is constructed by iterating through each time step and calculating the stock price at each node. To accelerate the computation and avoid factorial repetition on some reusable values, we use dynamic programming tricks where we start from the leaves of the binomial tree and use the children nodes' values to compute the value of the parent node.

2.1.3 Option Value Calculation

The option values at each node are calculated by discounting the expected value from the next time step back to the present. For American options, the intrinsic value is also considered for early exercise. Detailed core implementation of this project can be found in Appendix. For other full details of this project, please also refer to my GitHub repo: *https* : $//github.com/FrancisYu2020/IE498_project.git$

2.2 Convergence Analysis with Black-Scholes Model (Question 2)

The convergence of the binomial model to the Black-Scholes model is investigated by increasing the number of time steps (N) and comparing the binomial option prices to the Black-Scholes option price for a 1-year European call option. The price calculated by the binomial model with increasing time steps (N) and Black-Scholes model are shown in Fig 1 (left).

For this question we have $K = 100, S_0 = 100, r = 0.05, q = 0.04, \sigma = 0.2$. Using Black-Scholes formula we get the option price is $8.102643534463212 \approx 8.103$. From the figure we can see that as we increase the time steps N from 1 to 200, the binomial model option price oscillates around the Black-Scholes option price and eventually converges to that price. We also simulate our implemented binomial model with larger time steps. For computational efficiency we only compute the value for every 10 binomial steps and the results are plotted as Fig **??** shows. The error w.r.t the benchmark price (Black-Scholes price) is $0.000953 < 10^{-3}$. The N that makes the accuracy falls inside 10^{-3} is actually around 1910. For simplicity we show N = 2000 in the figure.

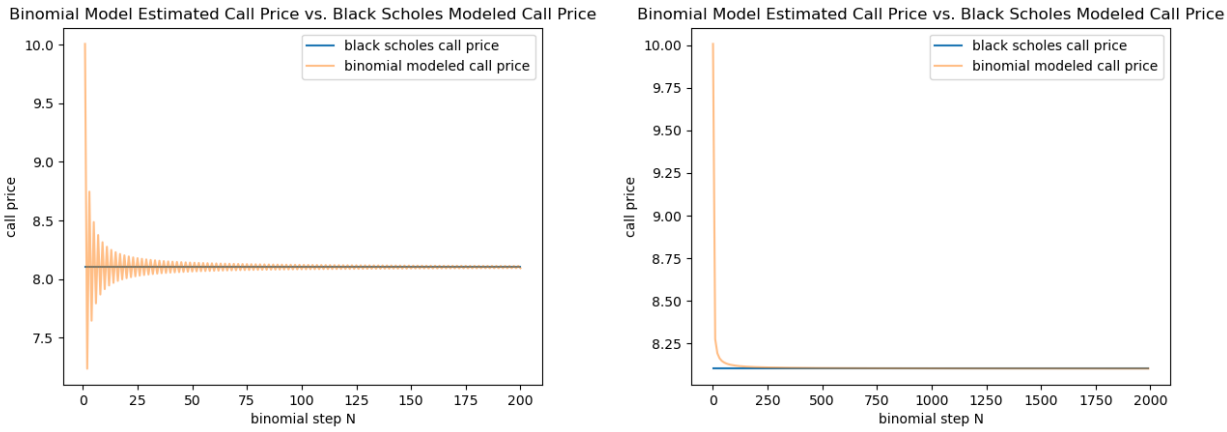

Figure 1: Figure of binomial model estimated European call price w.r.t increasing time steps vs. Black-Scholes option price. The original curve is the binomial model option price and the blue line is the Black-Scholes option price (**8.103**)

2.3 Early Exercise Boundary for American Put Options (Question 3)

The early exercise boundary for American options is explored by calculating the critical stock price S^* for different time to maturities and dividend yields. To get those values, we need to first have all the required parameters in the binomial model and make sure the time steps we use makes our model accurate enough. We first find the time steps that would satisfy the accuracy requirement under all variables. Since we are trying to find a δ for all time to maturity, so $\delta < \frac{1}{12}$.

To accelerate the grid search for convergence time steps, we start with $\delta = 0.01$, a moderate value that balances the number of trials required and large enough as a starter point, and decay δ by a factor of 1.1 whenever the convergence criterion does not satisfy our accuracy requirement for the current δ. Specifically,

$$\begin{aligned} N &= \frac{T}{\delta} \\ price1 &= Binomial(Option, K, T, S_0, \sigma, r, q, N, Exercise) \\ price2 &= Binomial(Option, K, T, S_0, \sigma, r, q, N+1, Exercise) \\ \epsilon &= |price1 - price2| \end{aligned}$$

In this question, if $\epsilon < 10^{-3}$, we regard the convergence criterion as satisfied. The reason we use 10^{-3} as the error tolerance is that from question 2, we observe that the binomial model estimated price oscillates around the benchmark price, and this trend holds for American option prices as well. So, by choosing $\epsilon < 10^{-3}$, the δ we get guarantees that the binomial model with N and $N+1$ time steps, which are very likely to be on the two sides of the actual benchmark price, deviates less than 10^-3 from the actual benchmark price. The advantage of this trick is that the heuristic is that most of the parameter groups being tested converge at large δ and we can reduce the number of "large N trials" with this automated process.

Another trick we use to accelerate this process is that we are using parallel programming techniques. The machine I use has many CPU cores, and using Python multiprocessing significantly accelerates this grid search procedure. Detailed implementation can be found in Appendix.

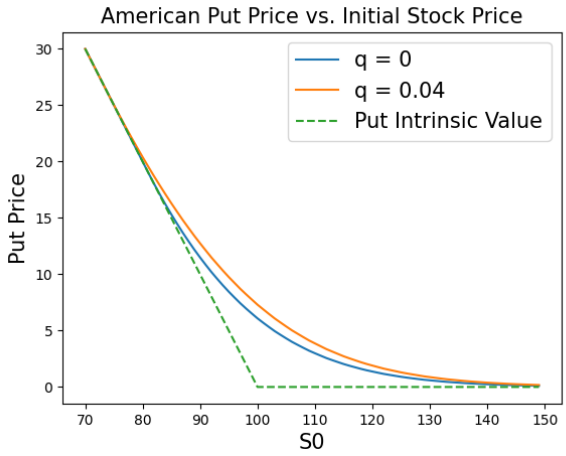

Figure 2: Figure of 12-month American put price vs. the initial stock price under different continuous yielding q

2.3.1 Question 3 Part 1 Results

With the above described method, we found that $\delta = 7.744 \times 10^{-5}$ is a small enough δ for all time to maturity and reasonable S_0 for both $q = 0$ and $q = 0.04$. For simplicity, we choose 7.5×10^{-5}. With that given δ, we plot the price of a 12-month put as a function of S_0 as Fig 2 shows. As the figure shows, when the initial stock price is somewhere around 80, with our given parameters, the American put price starts to deviate from the intrinsic value of this put option as the green curve shows. Also, when the initial stock price is large enough (somewhere around 150 in this question), the American put price goes to 0.

2.3.2 Question 3 Part 2 Results

Additionally, Fig 2 gives us a heuristic that $S^*(12)$ should be around 80 for both $q = 0$ and $q = 0.04$, which actually deviates a lot from slow convergence regions. By some exploration, we found that for both $q = 0$ and $q = 0.04$, and different time to maturity, $S^*(T)$ generally falls in the interval $[75, 95]$, where we find $\delta < 5.73 \times 10^{-4}$ would already be enough for the accuracy requirement aforementioned. This significantly saves the computation. In addition, we also use a larger δ first to get a coarse-grained estimation of $S^*(T)$ first, which we denote as $S^*(T)'$. Then based on that estimation, we search in the interval $[S^*(T)' - 0.5, S^*(T)' + 0.5]$ with 0.01 step size using the small enough δ, this tells us exactly where the cutoff value is. With this trick, we further significantly shortened the computation time.

The detailed results for this part are shown in Tab 1 and Fig 3. From the table and figure we can see that for the same time to maturity, the early exercise boundary (EEB) decreases as the continuous yielding increases. Also, for the same continuous yielding, the EEB decreases as the time to maturity increases.

2.4 Early Exercise Boundary for American Call Options (Question 4)

For this part we follow exactly the same procedure as described in Sec 2.3. The difference in the payoff function now uses $(0, S - K)^+$ to compute the call option payoff instead of put option.

T	$S^*(T)(q=0)$	$S^*(T)(q=0.04)$
1/12	91.32	88.89
2/12	88.92	85.47
3/12	87.36	83.21
4/12	86.19	81.50
5/12	85.24	80.10
6/12	84.46	78.94
7/12	83.78	77.93
8/12	83.20	77.04
9/12	82.67	76.26
10/12	82.21	75.55
11/12	81.78	74.90
12/12	81.40	74.31

Table 1: American put option early exercise boundary under different time to maturity and continuous yielding q

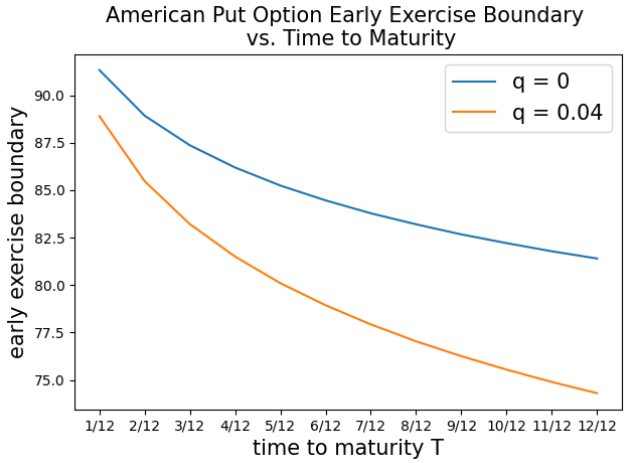

Figure 3: Figure of American put option early exercise boundary vs. time to maturity for different continuous yielding q

2.4.1 Question 4 Part 1 Results

Following the same procedure as described in Sec 2.3.1, we found that $\delta = 7.04 \times 10^{-5}$ is a small enough δ for all time to maturity and reasonable S_0 for $q = 0.04$ and $\delta = 7.744 \times 10^{-5}$ for $q = 0.08$. For simplicity, we choose 7×10^{-5}. With that given δ, we plot the price of a 12-month call as a function of S_0 as Fig 4 shows. As the figure shows, when the initial stock price is somewhere around 140, with our given parameters, the American call price starts to deviate from the intrinsic value of this call option as the green curve shows. Also, when the initial stock price is small enough (somewhere around 70 in this question), the American call price goes to 0.

2.4.2 Question 4 Part 2 Results

Following Sec 2.3.2, we repeat the same procedure and get the small enough δ to be $\delta < 7.04 \times 10^{-5}$. Similarly, we use $\delta = 7 \times 10^{-4}$ to coarsely find the estimated EEB and then use $\delta = 7 \times 10^{-5}$ to search within their neighbor values.

The detailed results for this part are shown in Tab 2 and Fig 5. From the table and figure

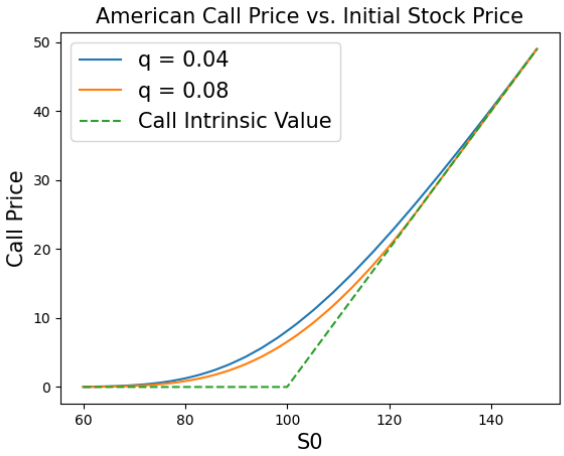

Figure 4: Figure of 12-month American Call price vs. the initial stock price under different continuous yielding q

we can see that for the same time to maturity, the early exercise boundary (EEB) decreases as the continuous yielding increases. Also, for the same continuous yielding, the EEB increases as the time to maturity increases.

T	$S^*(T)(q = 0.04)$	$S^*(T)(q = 0.08)$
1/12	125.06	110.54
2/12	128.74	113.93
3/12	132.27	116.27
4/12	135.46	118.08
5/12	138.28	119.57
6/12	140.80	120.84
7/12	143.07	121.95
8/12	145.14	122.94
9/12	147.05	123.82
10/12	148.82	124.61
11/12	150.46	125.34
12/12	152.02	126.02

Table 2: American call option early exercise boundary under different time to maturity and continuous yielding q

2.5 BBSR Method (Question 5)

In this section, we implement the BBSR method in Broadie and Detemple 1996. The major change of this method compared to the vanilla binomial model is that it uses Richardson Extrapolation, which is a mathematical technique used to improve the accuracy of numerical approximations by combining approximations with different step sizes. Specifically,If the option price computed using N steps is denoted as P_N and the price using 2N steps is denoted as P_{2N}, then using Taylor's expansion we know that the errors can be represented as:

$$P_N = P + \frac{A}{N} + \frac{B}{N^2} + O(\frac{1}{N^3})$$
$$P_{2N} = P + \frac{A}{2N} + \frac{B}{4N^2} + O(\frac{1}{8N^3})$$

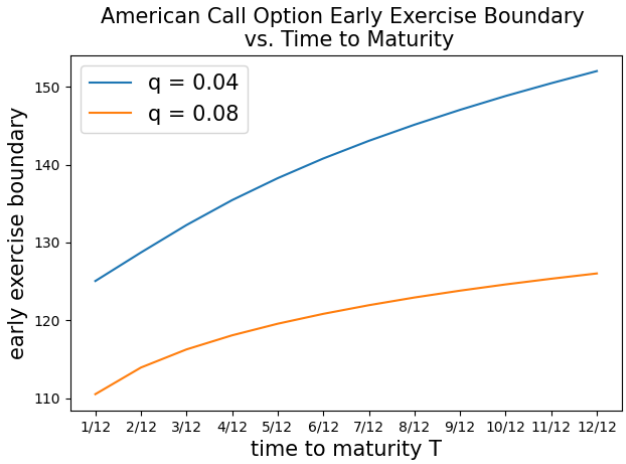

Figure 5: Figure of American call option early exercise boundary vs. time to maturity for different continuous yielding q

Where P is the true option price, A, B are constants. Then with Richardson Extrapolation we have:

$$P_{BBSR} = 2P_{2N} - P_N = P + O(\frac{1}{N^2})$$

With this simple trick, we significantly reduce the error of the model estimated price given large enough N. Implementation details can also be found in code Appendix.

Left figure in Fig 6 shows the simulation results using BBSR method. As we can see that instead of oscillating around the true option price as the binomial model does, BBSR method under-estimate the call option price in general. Also, we observed that at $N = 200$, the error of BBSR method is $3.006 \times 10^{-6} << 4.764 \times 10^{-3}$ which is the error of binomial model at $N = 400$ (we compare these two because BBSR method used $2N$ and for fair comparison we also need to compare with binomial model of time step $2N$). In addition, although computing 2 binomial model for one run provides some overhead in computation time, it is still on the same magnitude compared to binomial model with $2N$ steps, and more importantly, the accuracy is improved to almost ϵ^2 magnitude which is an attractive feature.

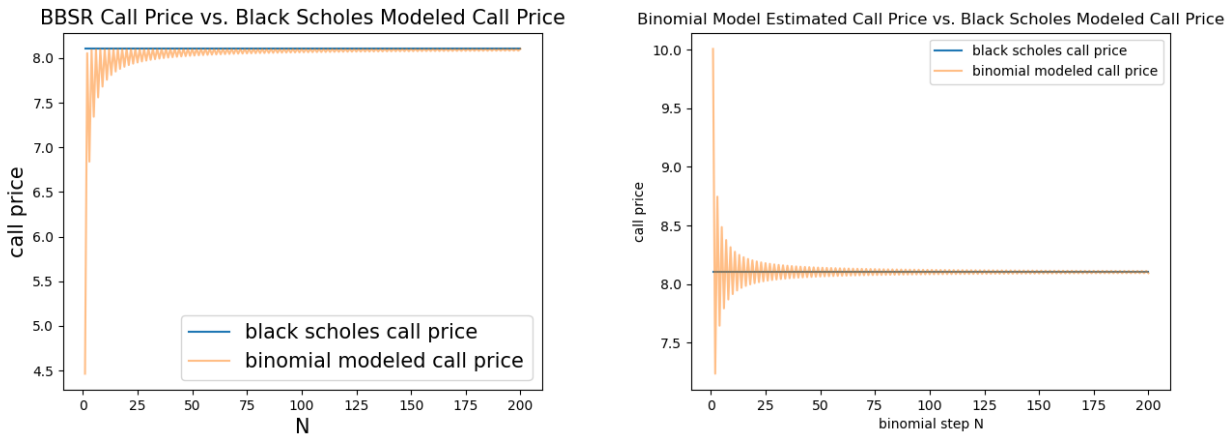

Figure 6: Figure of BBSR method estimated European call price vs. Black-Scholes modeled price (left) and vanilla binomial estimated European call price vs. Black-Scholes modeled price (right)

2.6 Binomial de-Americanization (Question 6)

I am interested in working on this part but I don't have enough time to finish this part by the given due. Would like to explore it more if possible.

2.7 NN de-Americanization (Question 7)

I am interested in working on this topic using neural networks for de-Americanization but I don't have enough time to finish this part by the given due. Would like to explore it more.

3 Discussion and Conclusion

3.1 Option Pricing

In this project, the option prices for European and American puts and calls are calculated using the binomial model. The results are compared to the Black-Scholes model for European options.

3.2 Convergence Analysis

From Sec 2.2 we concluded that the binomial model converges to the Black-Scholes European option price as the number of time steps (N) increases. And in Sec 2.3 and Sec 2.4 we also verified that given small enough δ, the binomial model will also converge to the American option prices with some error bound guarantee.

3.3 American Put Option

- **Changes in Put Prices**: As Fig 2 shows, when the dividend yield **increases**, the price of put options generally **increases**. The intuition behind is that dividends reduce the future expected price of the underlying asset. The more dividends a stock is expected to pay, the lower its expected price in the future. This makes put options more valuable, as they provide a right to sell the stock at a predetermined price (strike price) which becomes relatively more attractive as the stock price decreases due to dividends.

- **Changes in Early Exercise Boundary**: As Fig 3 shows, when the dividend yield **increases**, the early exercise boundary for American put option tends to **lower**. This means that with a higher dividend yield, it becomes optimal to exercise the put option earlier and at a higher stock price compared to a scenario with no dividends. The intuition behind this dependence on dividend yield effect is that:

    1. dividends are payouts to shareholders and, when paid, result in an immediate drop in the stock price by the amount of the dividend. Consequently, higher dividends lead to a lower expected stock price. This enhances the intrinsic value of put options, as the difference between the strike price and the expected lower stock price increases.
    2. For American options, which can be exercised at any time before expiration, the **opportunity cost of holding the option** rather than exercising it must be considered. With higher dividend yields, the opportunity cost of holding a put option increases because the holder misses out on receiving dividends. Therefore, it becomes optimal to exercise the option earlier to avoid missing out on the dividends.

3.4 American Call Option

- **Changes in Call Prices**: As Fig 4 shows, when the dividend yield **increases**, the price of call options generally **decreases**. The intuition behind is that the dividends reduce the future expected price of the underlying asset. Higher dividends mean that the stock is expected to drop more when the dividends are paid, which makes call options less valuable since the likelihood of the stock price rising above the strike price decreases.

- **Changes in Early Exercise Boundary**: As Fig 5 shows, when the dividend yield **increases**, the early exercise boundary for American call option tends to **lower** as well. This means that with a higher dividend yield, it becomes optimal to exercise the call option earlier and at a lower stock price compared to a scenario with lower dividends. Similarly to American put option, the intuition behind this dependence on dividend yield effect is that:

    1. Dividends are payouts to shareholders, and when paid, they result in an immediate drop in the stock price by the amount of the dividend. Consequently, higher dividends lead to a lower expected stock price. This reduces the intrinsic value of call options, as the difference between the strike price and the expected lower stock price decreases.
    2. Also, for American options, which can be exercised at any time before expiration, the opportunity cost of holding the option rather than exercising it must be considered. With higher dividend yields, the opportunity cost of holding a call option increases because the holder misses out on receiving dividends. Therefore, it becomes optimal to exercise the option earlier to capture the dividend, which pushes the early exercise boundary higher.

3.5 BBSR Method

Based on the previous experiments and results, here we give a more in-depth analysis and discussion of BBSR method:

- **Reduction of Bias:** By using two different step sizes and applying Richardson extrapolation, the BBSR method significantly reduces the bias that is inherent in the traditional binomial model. This reduction in bias leads to a more accurate estimation of the option price, especially for American options where the timing of exercise adds complexity.

- **Improved Convergence:** The convergence of the BBSR method to the true option price is faster compared to the traditional binomial model. This means that for a given level of accuracy, the BBSR method may require fewer steps, thereby improving computational efficiency. Faster convergence also implies that the method can achieve high accuracy with relatively fewer computational resources.

- **Versatility:** The BBSR method can be applied to both American and European options, making it a versatile tool for option pricing. It handles the early exercise feature of American options more effectively than traditional binomial models.

- **Practical Application:** In practice, the BBSR method is particularly useful for financial engineers and quantitative analysts who require precise option pricing models for hedging, trading, and risk management. The method's improved accuracy makes it a preferred choice for pricing complex derivatives and structured products.

- **Conclusion:** The BBSR method, developed by Broadie and Detemple, enhances the traditional binomial option pricing model by incorporating Richardson extrapolation. This approach reduces discretization error, improves accuracy, and offers faster convergence to the true option price. While it requires additional computation, the benefits of reduced bias and higher precision make it a valuable tool in the field of financial engineering and option pricing.

3.6 Binomial de-Americanization

I am interested in working on this part but I don't have enough time to finish this part by the given due. Would like to explore it more if possible.

3.7 NN de-Americanization

I am interested in working on this topic using neural networks for de-Americanization but I don't have enough time to finish this part by the given due. Would like to explore it more.

A Code Implementation

Listing 1: CRR Binomial Model Implementation

```
import numpy as np
import time
from scipy.stats import norm

def payoff(Option, K, S):
    assert Option in ['P', 'C'], "Only P and C accepted for option parameter"
    if Option == 'P':
        return max(0, K - S)
    else:
        return max(0, S - K)

def risk_neutral_probability(r, q, delta, u, d):
    return (np.exp((r - q) * delta) - d) / (u - d)

def Binomial(Option, K, T, S0, sigma, r, q, N, Exercise):
    '''
    Binomial model implementation for project question 1
    '''
    # start the timer
    start_time = time.time()

    # function to compute f(n, j)
    delta = T / N
    u = np.exp(sigma * np.sqrt(delta))
    d = np.exp(-sigma * np.sqrt(delta))
    p = risk_neutral_probability(r, q, delta, u, d)

    def payoff(n, j):
        if Option == 'C':
```

```
            # use call payoff function
            return max(0, S0 * u**j * d**(n - j) - K)
        elif Option == 'P':
            # use put payoff function
            return max(0, K - S0 * u**j * d**(n - j))
        else:
            raise NameError(f"Unrecognized option type {Option}!")

    # set up the memory buffer for dynamic programming
    # only use two vectors for memory efficiency
    memory = np.ones((2, N + 1))
    # initialize the leave node values
    for j in range(N + 1):
        memory[N % 2, j] = payoff(N, j)

    # main part for dynamic programming
    for n in range(N - 1, -1, -1):
        for j in range(n + 1):
            t0, t1 = n % 2, (n + 1) % 2
            if Exercise == 'A':
                memory[t0, j] = max(payoff(n, j), np.exp(- r * delta) * \
                (p * memory[t1, j + 1] + (1 - p) * memory[t1, j]))
            elif Exercise == 'E':
                memory[t0, j] = np.exp(- r * delta) * \
                (p * memory[t1, j + 1] + (1 - p) * memory[t1, j])
            else:
                raise NameError(f'Unrecognized option style {Exercise}!')

    option_price = memory[0, 0]
    end_time = time.time()

    return option_price, end_time - start_time
```

Listing 2: Black-Scholes Model for European Option Implementation

```
def black_scholes_call_price(S0, r, q, K, T, sigma):
    '''
    European Black-Scholes formula to compute call price
    Helper function for question 2
    '''
    d1 = (np.log(S0 / K) + (r - q + 0.5 * sigma**2) * T) / \
    (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return S0 * np.exp(- q * T) * norm.cdf(d1) \
    - K * np.exp(- r * T) * norm.cdf(d2)
```

Listing 3: δ Grid Search with Parallel Programming

```
# useful helper functions
def binomial_wrapper(args):
    Option, K, T, S0, sigma, r, q, N, Exercise = args
    ret = Binomial(Option, K, T, S0, sigma, r, q, N, Exercise)[0]
```

```
    return ret

def compute_delta(args):
    r, q, K, sigma, T, S0, tolerance = args
    delta = 0.01  # initial delta should be smaller than 1 / 12
    diff = np.inf
    while diff >= tolerance:
        delta /= 1.1
        curr_price = Binomial(option, K, T, S0, sigma, r, q, int(T / delta),
        next_price = Binomial(option, K, T, S0, sigma, r, q, int(T / delta) +
        diff = np.abs(curr_price - next_price)
        if diff < tolerance:
            return S0, delta
    return S0, delta

def find_delta(r, q, K, sigma, results_path='results.txt', write_mode='w', to
    if os.path.exists(results_path) and write_mode == 'w':
        print('Already saved results for question 3')
        with open(results_path, 'r') as f:
            print(f.read())
        return

#     tasks = [(r, q, K, sigma, T, S0, tolerance) for T in np.linspace(1/12,
    tasks = [(r, q, K, sigma, T, S0, tolerance) for T in np.linspace(1/12, 1,
    selected_delta = np.inf

    with mp.Pool(processes=mp.cpu_count()) as pool:
        for result in tqdm(pool.imap_unordered(compute_delta, tasks), total=l
            S0, delta = result
            selected_delta = min(selected_delta, delta)
            tqdm.write(f'S0 = {S0} converges at delta = {delta}')

    with open(results_path, write_mode) as f:
        f.write(f'selected delta for q = {q}: {selected_delta} ')
```

Listing 4: BBSR method implementation details

```
from utils import Binomial
def richardson_extrapolation(price_N, price_2N):
    '''
    BBSR uses Richardson extrapolation
    '''
    return 2 * price_2N - price_N

def BBSR(Option, K, T, S0, sigma, r, q, N, Exercise):
    # Calculate option prices using binomial trees with N and 2N steps
    price_N = Binomial(Option, K, T, S0, sigma, r, q, N, Exercise)[0]
    price_2N = Binomial(Option, K, T, S0, sigma, r, q, 2 * N, Exercise)[0]

    # Apply Richardson extrapolation
    BBSR_price = richardson_extrapolation(price_N, price_2N)
```

```
return BBSR_price
```

References

- Burkovska, O., Gass, M., Glau, K., Mahlstedt, M., Schoutens, W., & Wohlmuth, B. (2018). Calibration to American Options: Numerical Investigation of the De-Americanization Method. Quantitative Finance, 18(7), 1091-1113.
- Broadie, M., & Detemple, J. (1996). American Option Valuation: New Bounds, Approximations, and a Comparison of Existing Methods. The Review of Financial Studies, 9(4), 1121-1250.
- Gatheral, J., & Jacquier, A. (2013). Arbitrage-Free SVI Volatility Surfaces. Quantitative Finance, 14(1), 59-71.
- Lind, P., & Gatheral, J. (2023). NN De-Americanization: a Fast and Efficient Calibration Method for American-Style Options. Retrieved from `https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4616123`.